Use Cases

1. **Create a new project:** the developer creates a new project
2. **Load textures:** the developer loads base or custom textures for towers, enemies, strategic assets, GUI, background (map) and projectiles.
3. **Load font:** the developer loads base or custom font for GUI.
4. **Create an entity in project-global entities list:** the developer creates an entity (e.g. tower type, enemy type or some UI element) in project-global entities list.
5. **Load an entity to the level:** the developer loads an entity from project entities repository (e.g. tower spot, enemy wave) into currently opened level
6. **Remove an entity from the project:** the developer removes an entity from the project entities repository
7. **Remove an entity from the level:** the developer removes an entity from the level
8. **Configure wave entity:** the developer sets wave duration, next wave reference for wave, spawn rate, types of enemies for each enemy path involved in wave.
9. **Handle error:** the editor displays an error message.
10. **Get confirmation:** the editor asks for developers' confirmation before performing some action.
11. **Effect configuration:** the developer configures an effect to use it in damage types
12. **Damage type configuration:** the developer configures the type of damage.
13. **Tower type configuration:** the developer sets the damage type, attack range, attack cooldown, upgrade/sell system, active/inactive modes of tower entities.

14. **Enemy type configuration:** the developer sets enemy speed, HPs amount, texture and killing reward.
15. **Ability configuration:** configuring the ability (its type and parameters of this ability type)
16. **Grid and map layout setup**: configuring tile-based grid properties and background assets of the level map.
17. **Enemy path configuration:** defining and drawing the route trajectories and movement waypoints that enemies will follow. Setting reference to the HP counter, associated with this path
18. **Move entity on the map**
19. **Entity duplication:** cloning an existing entity along with all its configured fields.
20. **Group operations**: selecting multiple entities to move, duplicate or delete them in group.
21. **Save the project:** the editor saves all changes made to the project to its' files in directory there the project was created
22. **Import game:** the developer imports previously exported game so he can edit it inside of the editor.
23. **Launch the game:** the user launches the game using the generated executable.
24. **Place a tower:** the player selects a tower type and places it in an available position on the map.
25. **Sell a tower:** the player sells a selected tower and receives part of its cost back.
26. **Upgrade a tower:** the player upgrades a selected tower using available resources.
27. **Use an ability:** the player activates an available special ability during the game.
28. **Start the next wave:** the player starts the next enemy wave.

29. **Change game speed:** the player increases or decreases the game speed
30. **Kill an enemy**

Detailed Descriptions

1. Create a new project

Actors: developer
Goal: create a new project
Precondition/Context: The editor is running
Extensions:

- (step 4, see condition in step 3, **Get confirmation**)
- (steps 7,9, see condition in steps 7,9, **Handle error**)

Main success scenario:

1. The developer clicks the “New project” button in the editor menu
2. The editor creates a window, where developer can choose the project directory
3. The developer choose some project directory
4. If the project directory isn’t empty, call “Get confirmation” use case to ask for confirmation to clear the project directory
5. Else, got to step 8
6. If the developer cancel the operation, go to the step 3 and wait for another project directory chosen
7. Else, clear the project directory and go to step 8. If a file system error occurs, rollback all changes in file system made by game editor during project creation and call **Handle error** UC with appropriate message
8. The editor closes the window created on step 2

9. The editor creates appropriate directories and files inside the project directory. If a file system error occurs, rollback all changes in file system made by game editor during export and call **Handle error** UC with appropriate message
10. Editor initialize new in-memory project descriptor (with one empty level entity)
11. Editor opens the empty level associated with new project for editing

2. Load texture

Actors: developer
Goal: load base or custom texture for towers, enemies, strategic assets, GUI elements, map background or projectiles.
Trigger condition: developer opens one of texture entities for editing
Precondition/Context: The editor is running, a level is opened, there is at least one texture entity created
Extensions:

- (step 3, see condition in step 3, **Handle error**)

Main success scenario:

1. Developer presses a button to edit texture data property of texture entity
2. The editor creates a window, where developer can choose the image file
3. Developer chooses some image file. If file system error occurs or the selected file has an unsupported format, **Handle error** with appropriate error message, and abort importing
4. Editor loads the texture and displays it inside of an edited entities' property

3. Load font

Actors: developer
Goal: load base or custom font for GUI.
Trigger condition: developer opens one of font entities for editing
Precondition/Context: The editor is running, a level is opened, there is at least one font entity created
Extensions:

- (step 3, see condition in step 3, **Handle error**)

Main success scenario:

1. Developer presses a button to edit font data property of font entity
2. The editor creates a window, where developer can choose the font file
3. Developer chooses some font file. If file system error occurs or the selected file has an unsupported format, **Handle error** with appropriate error message, and abort importing
4. Editor loads the font to the edited entities' property

4. Create an entity

Actors: developer

Goal: create an entity

Trigger condition: developer press a button "new entity" in the editor menu

Precondition/Context: editor is running, a level is opened.

Main success scenario:

1. Developer selects the entity type (e.g. tower type, enemy type, some type of effect) from the presented list.
2. The editor adds a new entity with default property values to the project entities repository.

5. Load an entity to the level

Actors: developer
Goal: create an entity
Trigger condition: developer presses a button “+” corresponding to an entity container property of some of already loaded into the level entities (or game level as a first entity loaded when level is opened) editing menu
Precondition/Context: editor is running, a level is opened.
Main success scenario:

1. The editor opens a window with available entities of types permitted by the entity container property
2. Developer selects one of entities (e.g. EnemyWave or TowerSpot) from the presented list.
3. The editor places an entity inside of entity container property and updates the level entities tree to reflect this change
4. If the can be drawn on the map, the editor draws it

6. Remove an entity from the project

Actors: developer
Goal: remove an entity from the project entirely.
Trigger condition: developer presses “remove” button by one of the entities in project entities repository view
Precondition/Context: The editor is running, a level is opened.
Main success scenario:

1. The editor removes all references to this entity from all project entities

2. The editor removes the entity from the project entities repository

7. Remove an entity from the level

Actors: developer
Goal: remove an entity from the level.
Trigger condition: developer presses “remove” button by one of the entities in level entities tree
Precondition/Context: The editor is running, a level is opened.
Main success scenario:

1. The editor removes all references to this entity from all entities on the level
2. The editor removes the entity from the level entities tree

8. Configure wave entity

Actors: developer

Goal: configure enemy waves by setting wave duration and next wave for wave, adding some enemy paths to list of paths and setting spawn rate, enemy types for each enemy path.

Precondition/Context: editor is running, a level is opened, enemy paths are available, enemy entity types are available.

Main success scenario:

1. Developer opens one of wave entities’ editing menu
2. Developer adds an enemy path to list of paths involved in wave
3. Developer sets the enemy entity types for the added path
4. Developer sets the spawn rate for the added path
5. If developer wants to add one more enemy path to the wave, go to step 2

6. Developer sets wave duration and next wave reference (or none) for the wave
7. Developer saves the wave configuration

9. Handle error

Actors: developer

Goal: get the information about an error occurred during editing or configuration.

Precondition/Context: editor is running, the developer is performing an erroneous operation in the editor.

Main success scenario:

1. The editor creates a new window with:
    a. The error message displayed
    b. The “OK” button
2. The developer presses the “OK” button and the window closes

10. Get confirmation

Actors: developer
Goal: insure correctness of the operation that will be performed be the editor and possibly cancel it
Main success scenario:

1. The editor creates a new window with:
    a. The info about consequences of the operation that will be performed
    b. The “Proceed” button
    c. The “Cancel” button
2. If the developer presses “Proceed” button, the window closes and editor proceeds with the operation

3. Else, the window closes and editor doesn't proceed with the operation

11. Effect configuration

Actors: developer

Goal: configure an effect used for creating damage types.

Precondition/Context: editor is running, a level or is opened.

Main success scenario:

1. Developer opens one of effect entities' editing menu
2. Developer chooses effect type (damage, poisoning, slowness)
3. Developer configures parameters of the effect
4. Developer saves the effect configuration

12. Damage type configuration

Actors: developer

Goal: configure a type of damage used by game entities.

Precondition/Context: editor is running, a level or project is opened.

Main success scenario:

1. Developer opens one of damage type entities' editing menu
2. Developer adds references to effect entities associated with this damage type
3. Developer sets damage kind (projectile, instant reach, homing projectile)
4. Developer assigns texture for damage type
5. Developer saves the damage type configuration

13. Tower type configuration

Actors: developer
Goal: configure a type of tower that can be placed by player
Precondition/Context: the editor is running, textures for the tower and its projectiles have been loaded.
Main success scenario:

1. Developer opens one of tower type entities' editing menu
2. The developer assigns texture for the tower body and selects damage type;
3. The developer configures attack range and cooldown;
4. The developer add references to tower type entities to which this tower can be promoted and assigns price for such promotions
5. The developer previews the tower in context (range circle);
6. Developer saves the tower type configuration

14. Enemy type configuration

Actors: developer
Goal: configure an enemy type that can be spawned during wave
Precondition/Context: the editor is running, textures for the enemy have been loaded.
Main success scenario:

1. Developer opens one of enemy type entities' editing menu
2. The developer assigns texture for the enemy body
3. The developer configures enemy speed
4. The developer configures enemy HPs amount
5. The developer sets reward size for killing this enemy

6. The developer optionally adds multipliers for different effects that can affect enemies of this type (so the developer configures strengths and weaknesses)
7. Developer saves the enemy type configuration

15. Ability configuration

Actors: developer
Goal: configure an ability that can be used by player
Precondition/Context: the editor is running
Main success scenario:

1. Developer opens one of ability entities' editing menu
2. The developer configures parameters of the selected ability (parameters depend on the specific ability type, like healing ability or damage ability)

16. Grid and map layout setup

Actors: developer
Goal: configuring tile-based grid properties and background assets of the level map.
Precondition/Context: the editor is running, a level is opened, background textures have been loaded, there is at least one game map entity.
Main success scenario:

1. The developer opens one of game map entities for editing
2. The developer configures grid properties (cell size, map width/height in cells)
3. The developer clicks on button corresponding to the tile map property of game map entity
4. The editor creates a window for tile map editing
5. The developer assigns a texture to each of tiles

6. The developer saves; the editor writes tile map data to the game map tile map property and closes the window created at step 4.

17. Enemy path configuration

Actors: developer
Goal: define and draw route trajectories with waypoints that enemies follow path start to path end.
Precondition/Context: the editor is running, a level is opened, map layout is set up
Main success scenario:

1. Developer opens one of enemy path entities' editing menu, presses "draw a path" button and enters path editing mode
2. The developer places sequential waypoints on the grid to define the movement trajectory
3. When the arrangement is complete, the developer clicks the "complete editing" button and exits the special editor mode.
4. The engine changes enemy path entity state

18. Move entity on the map

Actors: developer
Goal: Move entity on the map
Precondition/Context: the editor is running, there is at least one entity on the map.
Main success scenario:

1. The developer selects an entity on the map;
2. The developer moves the cursor over the map and places the entity;
3. Game engine change this entities' state

19. Entity duplication

Actors: developer
Goal: clone an existing entity along with all its configured fields.
Precondition/Context: the editor is running, a level is opened, there is at least one entity in the project entities repository
Main success scenario:

1. The developer selects an entity in the entity repository
2. The developer invokes “duplicate”
3. Editor creates a copy of the entity with all property values copied

20. Group operations

Actors: developer
Goal: select multiple entities to move or delete them.
Precondition/Context: the editor is running, a level is opened, multiple entities exist on the map.
Main success scenario:

1. The developer selects multiple entities
2. The developer invokes a group action: move or delete
3. The editor changes state of the entities

21. Save the project

Actors: developer
Goal: save all changes made to the project.
Trigger condition: the developer presses the “save” button in the editor menu.
Precondition/Context: the editor is running, a level is opened
Extensions:

- (steps 1, 2, see condition in steps 1, 2, **Handle error**)

Main success scenario:

1. The editor overwrites all the entities that got changed since last save in file/files that store the entities repository.

If a file system error occurs, rollback all changes in file system made by game editor during export and call **Handle error** UC with appropriate message

2. The editor adds all new entities that got created since last save to file/files that store the entities repository. If a file system error occurs, rollback all changes in file system made by game editor during export and call **Handle error** UC with appropriate message

22. Import game

Actors: developer
Goal: edit previously exported game in the editor
Trigger condition: the developer presses the “import” button in the editor menu.
Precondition/Context: the editor is running.
Extensions:

- (step 4, see condition in step 4, **Handle error** with appropriate message)

Main success scenario:

1. The editor creates a window, there developer can choose the directory from which to import the game
2. The developer choose some directory
3. The editor reads game files in the directory
4. If a file system error occurs, game files have invalid format or some game file is corrupted (e.g. have null references), call **Handle error** UC with appropriate message, after that the game editor waits for the user to choose another directory or exit.
5. Else the editor opens the project for editing

23. Launch the game

Actors: player

Goal: launch the game
Trigger condition: the player executes game engine executable
Precondition/Context: the player has game engine installed, the player has the game files exported by the game editor
Main success scenario:

1. The game engine creates a window, there player can choose the directory with game files exported be the game editor
2. Player chooses some directory
3. The engine reads game files from the chosen directory.
4. If a file system error occurs, game files have invalid format or some game file is corrupted (e.g. have null references), the engine displays appropriate error message and waits for the user to choose another directory or exit.
5. Else the game engine starts the game

24. Place a tower

Actors: player
Goal: place a tower of specific type in an available position on the map to defend against enemies.
Trigger condition: player opens the tower selection UI
Precondition/Context: game is running; the map grid is visible; there is at least one empty tower spot
Main success scenario:

1. Player clicks on an empty tower spot
2. A tower actions menu appears near the tower, the upgrade button displays as active, the "sell" button is displayed as inactive
3. Player clicks on “upgrade tower” button in the tower actions menu

4. The tower actions menu closes and a tower upgrade menu with a list of available upgrade options opens. If player have sufficient amount of money for the specific upgrade, the button “Upgrade”, corresponding to it displayed as active, else it’s displayed as inactive
5. Player clicks the specific active "Upgrade" button corresponding to the desired upgrade option.
6. The upgrade tower menu closes and new tower appears in the spot

25. Sell a tower

Actors: player
Goal: remove a placed tower from the map and receive a portion of its cost back
Precondition/Context: game is running, at least one tower is placed on the map
Main success scenario:

1. Player clicks on a placed tower
2. A tower actions menu appears near the tower
3. Player clicks the "sell" button in the tower actions menu
4. The tower actions menu closes and player gets money

26. Upgrade a tower

Actors: player
Goal: improve a selected tower's stats
Precondition/Context: game is running
Main success scenario:

1. Player clicks on a placed tower

2. A tower actions menu appears near the tower
3. If this tower has any upgrade options, the upgrade tower button is displayed as active.
4. Else, the upgrade tower button is displayed as inactive and player cannot proceed with upgrading (**end of UC**)
5. Player clicks on “upgrade tower” button in the tower actions menu
6. The tower actions menu closes and a tower upgrade menu with a list of available upgrade options opens. If player have sufficient amount of money for the specific upgrade, the button “Upgrade”, corresponding to it displayed as active, else it’s displayed as inactive
7. Player clicks the specific active "Upgrade" button corresponding to the desired upgrade option.
8. Players’ money decreased by cost of the upgrade
9. The upgrade tower menu closes and tower changes its' type

27. Use an ability

Actors: player
Goal: damage the enemies
Precondition/Context: game is running, the ability is not on cooldown
Main success scenario:

1. Player clicks on the ability icon in the UI
2. If the ability requires targeting, player clicks on the target location on the map to confirm activation.
3. Ability activates

28. Start the next wave

Actors: player
Goal: make progress in completing a game level
Trigger condition: player clicks the “Call Next Wave” button
Precondition/Context: game is running; the previous wave ended
Main success scenario:

1. Player clicks the "Start Next Wave" button in the UI or presses the designated hotkey for starting the next wave (e.g., Spacebar or 'N').
2. The next wave starts

29. Change game speed

Actors: player
Goal: increase or decrease the simulation speed of the game.
Precondition/Context: game is running
Main success scenario:

1. Player clicks the increase or decrease button in the UI or presses the designated hotkeys to cycle through or set the game speed multiplier (e.g., '+', '-', or '1', '2', '3')
2. The game speed multiplier changes to chosen value

30. Kill an enemy

Actors: player
Goal: earn money and protect strategic asset (players’ HPs)
Precondition/Context: game is running, there is at least one enemy on the map
Main success scenario:

1. Tower or player (with ability) deliver damage to the enemy
2. Enemies’ HPs lower to zero
3. Enemy disappears from the map

4. The player earns some sum of money (depending on game rules)